

MPC for Constrained Joint-Space Motion on a UR7e Robot Arm for Warehouse Sorting

Parham Sharafoleslami, Ben Finch, Sharaf Hossain

December 19, 2025

[YouTube Link](#) [GitHub Link](#) [Slides Link](#)

Abstract—Warehouse sorting pick-and-place operations present a challenging control problem, as the robot arm is required to maintain smooth, collision-free motion under strict joint workspace, and safety constraints. This project proposes the design of a Model Predictive Controller (MPC) for constrained motion planning on a UR7e robot arm to perform warehouse-style sorting tasks. Following object detection and classification using an RGB-D vision pipeline, inverse kinematics is used to compute target joint configurations. Next, a receding-horizon MPC controller generates feasible joint-space trajectories. A discrete-time kinematic prediction model is deployed to enforce joint limits, bounded joint increments, obstacle avoidance, and table clearance constraints, while allowing intermediate orientation freedom and enforcing terminal orientation softly. The proposed controller is evaluated in both MuJoCo simulation and hardware experiments on the UR7e arm, where the controller demonstrated reliable convergence to target drop-off locations and consistent satisfaction of safety constraints. Solver performance remains within the control sampling period, which supports real-time replanning for constrained manipulation. Overall, the results showcase the functionality of joint-space MPC for perception-driven warehouse sorting tasks under realistic environmental and computational constraints.

I. INTRODUCTION

Automated warehouse sorting is an important problem in modern logistics and manufacturing, where robotic systems are required to perform reliable pick-and-place operations in constrained environments. These tasks require motion planning that respects joint limits, velocity bounds, and workspace constraints while maintaining responsiveness to perception updates and computational limitations. MPC is well-suited to this setting because it formulates motion generation as a constrained, finite-horizon optimization problem and allows the controller to anticipate constraint violations before they occur. In this project, MPC is applied within a perception-driven manipulation pipeline to generate feasible joint-space trajectories for a UR robot arm performing constrained pick-and-place motions typical of warehouse sorting scenarios.

A. Related Work

(MPC) has been applied to motion planning and control problems that require explicit handling of state, input, and safety constraints. Receding-horizon optimization can incorporate actuator limits and obstacle avoidance through inequality constraints, making MPC suitable for operation in confined environments [1]. In robotic manipulation, MPC has been used to generate constrained, obstacle-avoiding trajectories for robot arms operating in cluttered workspaces, with real-time implementations demonstrated on industrial robotic manipulators [2], [3]. Joint-space MPC formulations provide a prac-

tical balance between constraint handling and computational tractability, enabling direct enforcement of joint limits while maintaining smooth motion generation for manipulation tasks [4].

B. Contribution

Building on these approaches, this project implements a joint-space MPC pipeline for constrained pick-and-place manipulation. An RGB-D camera (Intel RealSense) captures data used for object and obstacle localization, inverse kinematics maps desired end-effector poses to joint-space targets, and a receding-horizon MPC controller generates feasible joint trajectories subject to joint position and velocity constraints. The emphasis is on system integration and real-time feasibility rather than novel control formulations, employing solver warm-starting strategies to achieve control loop frequencies suitable for hardware deployment. Evaluation is carried out in MuJoCo simulation and hardware experiments on a UR7e manipulator, representative of warehouse sorting scenarios.

II. SYSTEM OVERVIEW

All experiments were conducted using an UR7e robotic manipulator provided by the EECS/ME206A laboratory. The UR7e is an articulated 6-degree-of-freedom robot equipped with high-resolution joint encoders and an internal low-level control stack capable of executing joint position and velocity commands. In this work, the robot was controlled exclusively through its joint velocity interface, which provides reliable tracking performance while abstracting actuator-level dynamics. Thus, allowing for safe operation in close proximity to the workspace.

The experimental task was designed to imitate a simplified warehouse sorting scenario. Small cubic objects placed on a tabletop represent items to be picked and sorted, while rectangular bins marked on the table surface define designated drop-off zones. Objects were manually placed at varying initial locations to introduce variability in trials. The end effector was equipped with a parallel-jaw gripper to enable reliable grasping and transport of objects.

A static obstacle, implemented as a rigid plastic storage container, was positioned between the robot and the target regions to enforce nontrivial collision-avoidance behavior. This obstacle represents shelving or other infrastructure commonly encountered in warehouse environments. To perceive the workspace, an Intel RealSense RGB D435i camera was mounted overlooking the table, providing point-cloud and

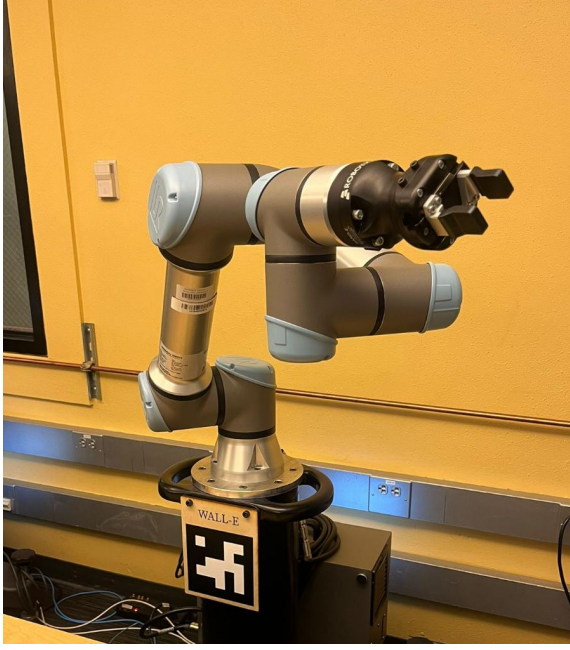


Fig. 1. UR7e Robot Arm in EECS/ME 206A Lab.

depth information used to estimate the positions and axis-aligned bounding boxes (AABBs) of obstacles and objects in the scene.

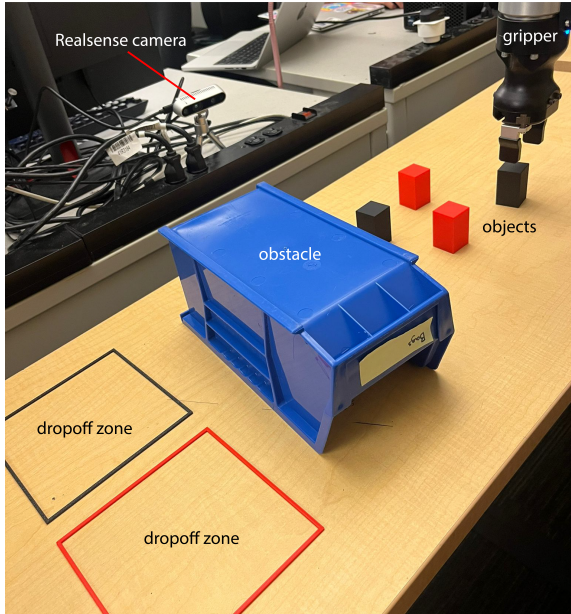


Fig. 2. Setup of Objects, Obstacles and Target Basket.

III. SYSTEM MODELING

A. State and Input

The MPC operates in joint space. The state is the 6-DoF arm configuration $q_k \in \mathbb{R}^6$ (joint angles) at discrete step k . The control input is a bounded joint position increment $\Delta q_k \in \mathbb{R}^6$,

which is applied over a fixed sample period Δt . The discrete-time model is:

$$q_{k+1} = q_k + \Delta q_k, \quad (1)$$

where $\Delta t = 0.08$ s and prediction horizon $N = 30$. Per-step bounds $|\Delta q_{k,i}| \leq 0.15$ radians reflect the position-increment interface of the controller (e.g a bounded average joint velocity over each sample). This formulation keeps the discrete dynamics simple while allowing hard joint and workspace constraints to be enforced in the optimization problem.

B. Prediction Model

System evolution over the horizon uses the same discrete-time kinematic model:

$$q_{k+1} = q_k + \Delta q_k, \quad (2)$$

assuming piecewise-constant joint increments over each sampling interval. We use $\Delta t = 0.08$ s and a prediction horizon of $N = 30$. This simple model keeps the optimization fast enough for real-time replanning while remaining sufficient for the joint-space motion tasks considered here.

C. Constraints

(i) Joint Limits and Step Bounds: Joint positions are bounded by the physical limits: $|q_{k,i}| \leq 2\pi$. To limit aggressive motions and approximate actuator constraints, per-step position increments are limited to:

$$-0.15 \leq \Delta q_{k,i} \leq 0.15 \text{ rad}, \quad \forall i, k$$

This reflects the position-increment interface at $\Delta t = 0.08$.

(ii) Obstacle avoidance: Obstacle avoidance is enforced in task space using a geometric approximation of the end effector and grasped object. The combined geometry is represented by three proxy spheres centered at $[0, 0, 0]$, $[0, 0, 0.02]$, and $[0, 0, 0.03]$ m in the tool frame, with inflated radii $r_i \in [0.03, 0.05]$ m. At each prediction step and for each proxy sphere, a minimum distance constraint is imposed:

$$d(c_i(q_k), \mathcal{B}) \geq r_i, \quad (3)$$

where $c_i(q_k)$ denotes the sphere center computed via forward kinematics and $d(\cdot)$ is the distance to the perceived obstacle's axis-aligned bounding box (AABB) expressed in the robot base frame. Let the obstacle AABB bounds be $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$, and let $c = [c_x, c_y, c_z]^T$ be a proxy-sphere center in the base frame. The Euclidean distance from a point to the box is computed as

$$\begin{aligned} \delta_x &= \max\{x_{\min} - c_x, 0, c_x - x_{\max}\}, \\ \delta_y &= \max\{y_{\min} - c_y, 0, c_y - y_{\max}\}, \\ \delta_z &= \max\{z_{\min} - c_z, 0, c_z - z_{\max}\}, \\ d(c, \mathcal{B}) &= \sqrt{\delta_x^2 + \delta_y^2 + \delta_z^2}. \end{aligned} \quad (4)$$

Equivalently, we define the instantaneous obstacle clearance margin

$$m_{\text{obs}}(q) \triangleq \min_{i \in \{1, 2, 3\}} (d(c_i(q), \mathcal{B}) - r_i), \quad (5)$$

so collision-free motion corresponds to $m_{\text{obs}}(q) \geq 0$ and a safety buffer of $\epsilon_{\text{obs}} > 0$ corresponds to $m_{\text{obs}}(q) \geq \epsilon_{\text{obs}}$.

(iii) Table clearance: A vertical floor constraint prevents collisions with the table, softened via slack to maintain feasibility under table-height uncertainty. The effective floor is phase-dependent (e.g. strict table height on descent and raised to obstacle top on return to force overflight). The use of slack enables soft constraints to be implemented. Let $z_{\text{floor}}(\phi)$ denote the phase-dependent effective floor height and let $\epsilon_{\text{tab}} > 0$ be the clearance margin. Using the proxy-sphere centers, the table constraint is written as

$$m_{\text{tab}}(q, \phi) \triangleq \min_i (c_i^z(q) - (z_{\text{floor}}(\phi) + \epsilon_{\text{tab}})) \geq -s_{\text{tab}}, \quad (6)$$

where $s_{\text{tab}} \geq 0$ is a slack variable penalized in the cost.

(iv) Terminal orientation constraint: A down-facing orientation constraint on the tool frame is enforced only at the terminal step of the horizon. This constraint is imposed softly with slack, this enables intermediate configurations to remain unconstrained in orientation so that the optimizer may generate safer collision-free detours.

Hence, these constraints define the feasible set for the MPC optimization problem. The associated cost function, described in Section IV, penalizes end-effector position error, control effort, and terminal error, with an increased terminal weight to encourage convergence to the desired configuration to drop objects to the target destination.

IV. MPC FORMULATION

Given a desired end-effector pose, an inverse kinematics (IK) solver is first used to compute a target joint configuration q^* . Starting from the current measured joint state, the MPC computes a joint-space trajectory that drives the system toward q^* while respecting kinematic, collision, and safety constraints. At each control cycle, a finite-horizon optimal control problem is solved and only a portion of the optimized input sequence is executed before replanning, this results in a closed-loop receding-horizon controller.

A. Cost Function

The MPC objective is designed to encourage accurate end-effector positioning, smooth joint motion, and reliable convergence to the goal configuration. The total cost is given by

$$J = \sum_{k=0}^{N-1} \|p(q_k) - p^*\|_{Q_p}^2 + \|\Delta q_k\|_R^2 + \|p(q_N) - p^*\|_{Q_{p,N}}^2 + w_{\text{face}} s_{\text{face}}^2 + w_{\text{tab}} \sum_k s_{\text{tab},k}^2, \quad (7)$$

where $p(q_k)$ denotes the end-effector position obtained via forward kinematics and p^* is the desired end-effector position.

The stage cost penalizes Cartesian position error and joint step magnitude, with weighting matrices $Q_p = I$ and $R \ll Q_p$, encouraging goal-directed motion and limiting aggressive joint updates. A larger terminal weight $Q_{p,N} = 10Q_p$ is applied to encourage convergence at the end of the horizon. Orientation tracking along the trajectory is intentionally omitted

to allow flexible obstacle avoidance. However, a down-facing terminal orientation is enforced softly using the slack variable s_{face} . Similarly, table-clearance constraints are softened using slack variables $s_{\text{tab},k}$ to preserve feasibility under perception uncertainty.

B. Finite-Horizon Optimization Problem

At each control step, the MPC solves the following nonlinear finite-horizon optimization problem:

$$\min_{\{q_k, \Delta q_k\}, s} J \quad (8)$$

$$\text{s.t. } q_{k+1} = q_k + \Delta q_k, \quad k = 0, \dots, N-1, \quad (9)$$

$$q_{\min} \leq q_k \leq q_{\max}, \quad (10)$$

$$\Delta q_{\min} \leq \Delta q_k \leq \Delta q_{\max}, \quad (11)$$

$$d(c_i(q_k), \text{AABB}) \geq r_i, \quad \forall i, k, \quad (12)$$

$$z_{\text{table}} + \text{margin} \leq c_i^z(q_k) + s_{\text{tab},k}, \quad (13)$$

$$\text{terminal down-facing constraint via } s_{\text{face}}, \quad (14)$$

$$s_{\text{face}} \geq 0, \quad s_{\text{tab},k} \geq 0. \quad (15)$$

Here, $c_i(q_k)$ denotes the center of proxy sphere i computed via forward kinematics, with corresponding radius $r_i \in [0.03, 0.05]$ m. The function $d(\cdot, \text{AABB})$ computes the minimum distance between each proxy sphere and the perceived obstacle axis-aligned bounding box expressed in the robot base frame. The table clearance constraint enforces a minimum vertical separation between the robot geometry and the estimated table surface, while allowing controlled violation through penalized slack variables when necessary.

C. Receding Horizon Implementation

The optimization problem is solved in a receding-horizon manner. At each control cycle, the MPC is initialized with the current measured joint configuration and the latest perceived obstacle geometry. Rather than applying only the first control input, the first three joint increments, which corresponds to approximately 0.15 s of motion are executed before replanning occurs. This choice balances responsiveness with reduced solver invocation frequency.

The MPC loop terminates once the end effector is within 3 cm of the desired target position or when a predefined maximum number of solver iterations is reached, which was set to 40 solves in our implementation. Closed-loop replanning was essential for maintaining obstacle clearance and ensuring convergence in the presence of perception updates, model mismatch, and environmental uncertainty.

V. RESULTS AND DISCUSSION

The proposed joint-space MPC framework was evaluated in simulation using a physics engine (MuJoCo) to assess closed-loop convergence and computational feasibility in a warehouse-style sorting environment. The scene includes a tabletop workspace, a static obstacle modeled as an axis-aligned bounding box (AABB), and a target drop-off location as described in Section III. The simulated environment is shown in Fig. 3.

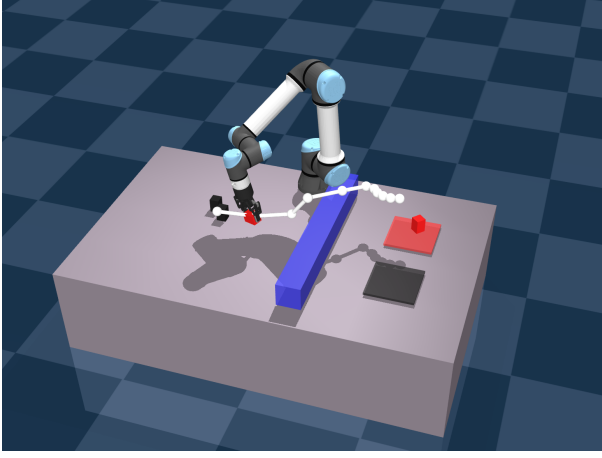


Fig. 3. MuJoCo simulation of a warehouse-style sorting task using the UR7e robotic arm. The setup includes graspable objects and a static obstacle positioned between the robot and the target region.

Fig. 4 shows the executed end-effector trajectory in Cartesian space for a representative transport phase. Starting from the initial pose, the MPC generates a smooth path that intentionally detours to maintain clearance from the obstacle before converging toward the target drop-off position. This behavior is a direct result of the receding-horizon formulation in Section IV, where at each control cycle, the optimizer trades off goal progress against collision avoidance and table safety constraints, while intermediate tool-frame orientation is left unconstrained (terminally enforced with slack). As a result, the controller is not forced into excessively restrictive motions and can produce obstacle-avoidant paths while still converging reliably.

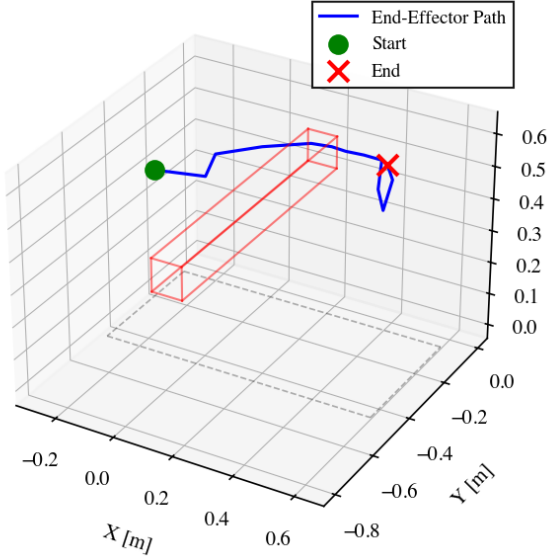


Fig. 4. Executed end-effector Cartesian trajectory (blue) in the presence of the obstacle AABB (red). The MPC detours around the obstacle before converging to the goal.

To quantify convergence, Fig. 5 shows the end-effector position error norm over time. The error decreases steadily from an initial offset on the order of ≈ 0.7 m to within the 3 cm stopping tolerance. Small non-monotonic fluctuations near convergence are expected in closed-loop MPC due to replanning and the use of softened constraints, but the controller remains stable and reaches the goal without violating safety constraints.

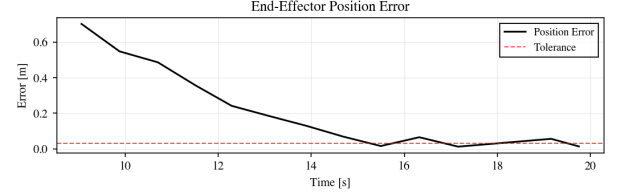


Fig. 5. End-effector position error norm over time (black). The dashed line indicates the 3 cm goal tolerance used to terminate the MPC loop.

Constraint satisfaction is demonstrated through both joint-space and workspace metrics. Fig. 6 shows the commanded joint increments over time, which remain within the imposed per-step bounds of ± 0.15 rad, matching the actuation and discretization assumptions in Section III-A. This confirms that the MPC does not demand aggressive joint motion and instead produces smooth, bounded updates consistent with the position-increment interface.

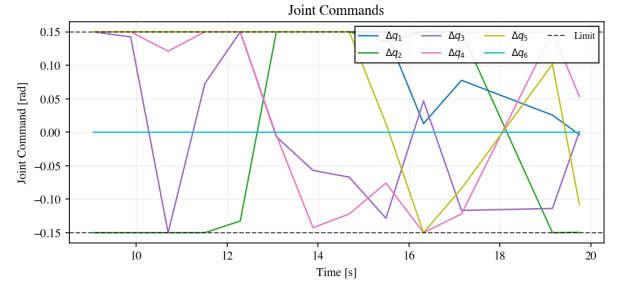


Fig. 6. Joint increment commands Δq_i over time. Dashed lines denote the enforced per-step bounds of ± 0.15 rad.

Workspace safety constraints are validated in Fig. 7 and Fig. 8. The obstacle clearance margin (minimum distance between the proxy-sphere geometry and the obstacle AABB, minus the sphere radius) remains strictly positive and above the configured safety margin throughout execution, indicating collision-free behavior even during the detour. Similarly, the table clearance margin remains well above zero, demonstrating that the floor constraint prevents table contact during transport. Together, these plots provide direct evidence that the constraint set is respected during closed-loop execution, while the use of soft constraints helps retain feasibility in proximity to obstacles and near the table surface.

Finally, Fig. 9 summarizes solver performance. The average solve time is ≈ 11.7 ms with occasional higher-cost iterations, and remains below the control sample period used in this evaluation ($\Delta t = 0.08$ s). This indicates that the optimization can

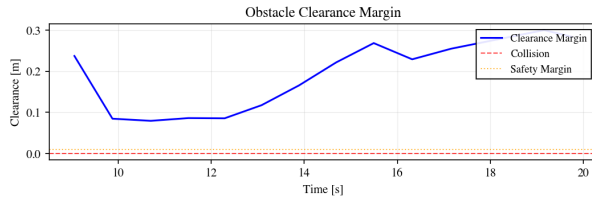


Fig. 7. Minimum obstacle clearance margin over time (blue). The dashed line at zero indicates collision; the dotted line indicates the configured safety margin.

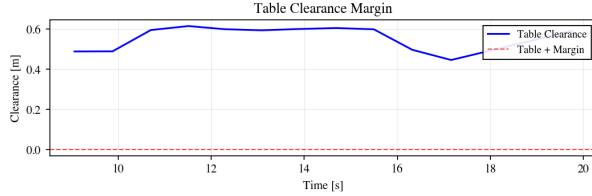


Fig. 8. Table clearance margin over time. The trajectory maintains a positive clearance relative to the table height plus margin.

be solved in real time for the considered horizon and constraint set, which supports the practical feasibility of receding-horizon re-planning for constrained manipulation tasks.

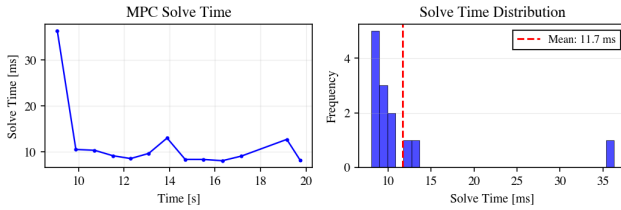


Fig. 9. MPC solve time per iteration (left) and solve time distribution (right). The mean solve time is approximately 11.7 ms, enabling real-time replanning at $\Delta t = 0.08$ s.

Overall, the results demonstrate that joint-space MPC can generate smooth, goal-directed motion while explicitly enforcing joint-step bounds, obstacle avoidance, and table safety constraints. The closed-loop replanning structure is key as it enables nontrivial detours around obstacles (Fig. 4) while still converging to the goal with bounded control actions (Fig. 5–6) and maintaining positive safety margins in the workspace (Fig. 7–8). Observed limitations include sensitivity to perceived obstacle geometry and occasional increases in solve time for more constrained configurations, which motivates future work on perception robustness and warm-starting strategies.

VI. CONCLUSION

This paper presented a constrained joint-space MPC framework for robotic warehouse sorting tasks. By integrating perception, kinematics, and receding-horizon control, the system generates feasible, collision-avoiding trajectories under realistic constraints. The results demonstrate the practical viability of MPC for constrained manipulation, while highlighting areas

for future improvement such as perception robustness and computational efficiency.

REFERENCES

- [1] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*, 1st ed. Cambridge University Press, 2017.
- [2] M. Rubagotti, T. Taunyazov, and O. Bukeikhan, “Semi-autonomous robot teleoperation with obstacle avoidance via model predictive control,” in *Proceedings of Robotics: Science and Systems (RSS)*, Freiburg im Breisgau, Germany, 2019.
- [3] A. Mavrommati, C. Osorio, R. Valenti, A. Rajhans, and P. Mosterman, “An application of model predictive control to reactive motion planning of robot manipulators,” in *Proceedings of the IEEE International Conference on Automation Science and Engineering (CASE)*, 2021, pp. 915–920.
- [4] M. Bhardwaj, B. Sundaralingam, A. Mousavian, and N. Ratliff, “Storm: An integrated framework for fast joint-space model predictive control for reactive manipulation,” in *Proceedings of the 5th Conference on Robot Learning (CoRL)*, London, United Kingdom, 2021.